



RIMO

안전데이터 기반 안심귀가 및 보호자 연동 서비스

“가장 빠른 길이 아닌, 더 안심할 수 있는 길”

목차

RIMO 프로젝트 발표 구성

01

프로젝트 배경

벤치마킹 · 문제 정의 · 목표

03-07

02

서비스 기능

서비스 링크 · 주요 기능 요약

08-12

03

시스템 아키텍처

전체 구조 · AWS

13-16

04

시스템 구성 및 운영

EKS · 보안 · CI/CD · 모니터링 · IaC

17-30

05

결과 및 팀 소개

기술 스택 · 기대효과 · 팀원 · 참고자료

31-35

SECTION 01

프로젝트 배경

왜 RIMO가 필요한가

벤치마킹과 사회적 배경을 바탕으로 해결해야 할 문제를 정의합니다.



01

기획 배경 및 문제점 인식

객관적 통계와 언론 보도로 확인한 야간 보행 불안

13세 이상 25% "한국 안전하지 않아"...여성 45% "밤길 무서워"(종합)

송고 2024-11-12 16:49

세 줄 요약

구글에서 선호하는 출처로 추가

13세 이상 인구 중 밤에 혼자 걸을 때 불안하다고 느낀 비중은 30.5%로 2년 전(29.6%)보다 0.9%p 상승했다. 밤길에 불안을 느낀다고 답한 비중은 2014년 42.6%를 기록한 뒤로 꾸준히 하락했지만 올해 상승세로 돌아섰다.

성별로 보면 여성의 44.9%, 남성은 15.8%가 야간 보행 때 불안을 느낀다고 답했다.

핵심 통계

30.5%

전체 인구

44.9%

여성

2년간 29.6%에서 30.5%로 +0.9%p

야간 보행 주요 불안 요인 분석

인적 드문 환경 26.4%

안전 시설 부족 16.9%

결론 및 시사점

환경적 요인 · 안전시설 부족 → 야간 보행 불안 증가
안전환경 정보 기반 귀가 지원 필요

안전 서비스 효과 및 사회적 근거

안전 서비스 도입 효과와 시민 체감 자료

서울 시민 안전 지키는 '안심헬프미'... 강력범죄 우려 속 주목

버튼 누르면 경고음, CCTV관제센터 신고, 경찰출동까지 체계적 시스템으로 위기 대응... "가지고만 있어도 안심"
20'24년 도입 후 청소년·여성 등 현재까지 11만개 배포

장경근

입력 2026.05.11 22:20 | 댓글 0

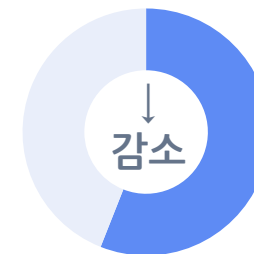
'안심헬프미'를 통한 신고 건수는 2024년 624건, 2025년 693건이었으며 올해는 5월 7일 기준 858건이 접수됐다. 경찰이 출동한 사례는 총 20여건('24년~'26년 5.7.)이다. 특히 작지만 강력한 경고음만으로도 주변에 위험을 알리는 효과가 있어 실제 위급 상황 대응에 도움이 되는 것으로 나타났다.

시민만족도 조사에서도 긍정적인 반응이 두드러졌다. 2024년 조사결과 '안심헬프미'를 받은 시민 73.2%가 만족한다고 답했고, 56%는 혼자 길을 걸을 때 두려움이 줄었다고 답해 서울시의 안전 정책이 실질적인 체감 효과로 이어지고 있음을 보여준다.

안전 서비스 이용 효과

서비스 이용 시 두려움 감소 비율

56.0%



안전 서비스 도입 후 두려움 감소 응답 비율

안전 서비스 이용을 통한 불안감 감소 효과 확인
→ 체감 안전성을 높이는 안전 서비스의 필요성 확인

벤치마킹을 통해 발전시킨 RIMO의 특징점

기존 서비스 분석을 통한 프로젝트 요구사항 구체화



서울 안심이 (서울 지역 서비스)

공공 관제센터 연계 및 긴급신고 중심

BENCHMARKING >



RIMO (서울·경기·인천 지역 서비스)

안전데이터 기반 경로 비교·분석 및 생성형 AI 설명 제공



긴급 신고 URL 공유

긴급 신고 발생 시 보호자에게 문자와 위치 확인 URL을 전송합니다.
보호자가 긴급자의 실시간 위치를 확인할 수 있습니다.

문자 내 URL 클릭



실시간 위치 확인



직관적인 목적지 표시

기존 지번·주소 중심의 목적지 표현 대신 사용자가 쉽게 이해할 수 있는 실제 장소명을 표시합니다.

서울특별시 용산구 한강대로 405



서울역



사용 리포트 / 대시보드

사용자의 이동 및 서비스 이용 정보를 리포트와 대시보드 형태로 시각화하여 확인할 수 있습니다.



프로젝트 목표

RIMO 서비스가 지향하는 3가지 핵심 가치



안전경로 지원

안전시설 데이터를 기반으로
안전한 이동 경로 선택 지원



실시간 보호

위치 공유와 이상 상황 감지를 통해
이동 중 사용자의 안전 지원



안정적 인프라

이중화된 Cloud Native 구조를 통해
안정적인 서비스 환경 구축

SECTION 02

서비스 기능

사용자 경험의 중심

복잡한 구현 화면 대신 링크와 핵심 기능을 명확하게 소개합니다.



RIMO 소개 웹사이트

서비스의 기능과 다운을 웹 사이트에서 한눈에!



서비스 소개 주요 기능 사용법 시연 영상

앱 다운로드

— 안전 귀가 서비스

늦은 귀갓길, 더 안심할 수 있게.

RIMO는 AI 안전경로 추천부터 안심친구 위치 공유, 이상 상황 확인과 보호자 알림까지 귀가 과정을 하나로 연결합니다.

QR로 앱 받기 →

기능 체험하기

✓ AI 안전경로 ✓ 실시간 위치 공유 ✓ 보호자 알림



<https://www.rimo-app.com/>



서비스 기능 소개

귀가 전·중·후를 하나로 연결하는 RIMO 앱의 6가지 기능

AI SAFE ROUTE

안심경로

01

목적지를 검색해 빠른길·AI 안전경로·대로변을 비교하고, 선택한 경로로 귀가 안내를 시작합니다.

- ✓ 목적지 검색 → 경로 비교
- ✓ AI 추천 이유 확인
- ✓ 귀가 진행 중 연속 경로 안내

LIVE LOCATION

안심친구

02

안심친구와 필요한 순간 위치를 공유하고, 친구의 현재 위치를 지도에서 확인합니다.

- ✓ 위치 공유 ON/OFF
- ✓ 친구 현재 위치 확인
- ✓ 이상 상황 알림 연결

SAFETY MAP

안심지도

03

현재 위치 주변의 CCTV·보안등·비상벨 등 공공 안전시설을 필요한 종류만 골라 확인합니다.

- ✓ 안전시설 한 종류씩 선택
- ✓ 현재 위치 중심 탐색
- ✓ 주변 안전 인프라 확인

EMERGENCY

긴급신고

04

긴급 상황에서 등록된 보호자에게 현재 위치와 위치 확인 링크를 빠르게 전달합니다.

- ✓ 보호자 알림
- ✓ 현재 위치 전달
- ✓ 지도 링크 제공

SIREN

꽹꽹이

05

긴급한 순간 경고음과 플래시를 즉시 작동시켜 주변에 위험 상황을 알리는 보조 기능입니다.

- ✓ 경고음
- ✓ 플래시
- ✓ 즉시 실행·중지

MY REPORT

사용 리포트

06

대시보드에서 귀가 요약을 보고, 기록 탭의 달력에서 날짜별 귀가 상세를 확인합니다.

- ✓ 귀가 요약 대시보드
- ✓ 달력에서 날짜 선택
- ✓ 선택 날짜 귀가 상세 확인

부가 서비스

위험을 먼저 감지하고, 보호자가 바로 찾아올 수 있도록



AUTO DETECTION

이상 상황 자동 감지

귀가 중 사용자의 이동 경로와 정차 상태를 분석해, 위험 신호를 자동으로 감지합니다.

감지하는 이상 상황

1 비활동 감지

귀가 중 일정 시간 이상 움직임이 없을 때

2 경로 이탈

선택한 안심경로에서 벗어났을 때

감지 후 대응 흐름

이상 감지



사용자 확인 알림



응답 없으면 보호자 알림



GUARDIAN ALERT

보호자 긴급 알림

● 실시간

긴급신고나 이상 감지 시, 보호자가 사용자의 위치를 실시간으로 보고 바로 찾아갈 수 있습니다.

1

긴급신고 · 이상 감지

버튼 한 번 또는 자동 감지로 시작

2

보호자에게 문자 + 위치 URL 전송

앱 설치 없이 링크만 누르면 확인

3

지도에서 사용자 위치 실시간 확인

이동하면 마커도 함께 이동

4

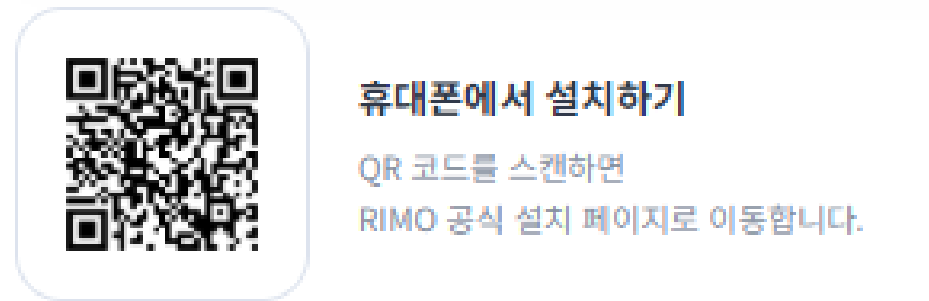
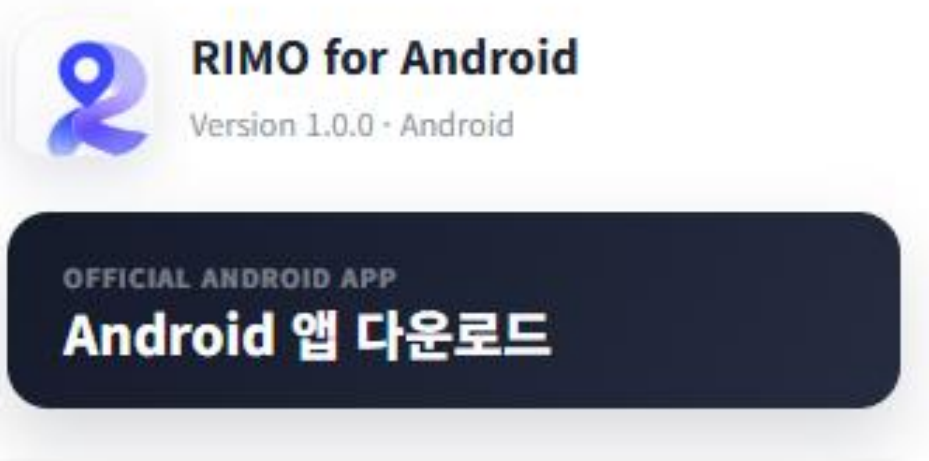
마커 클릭 → 바로 길찾기

사용자 위치까지 경로 안내

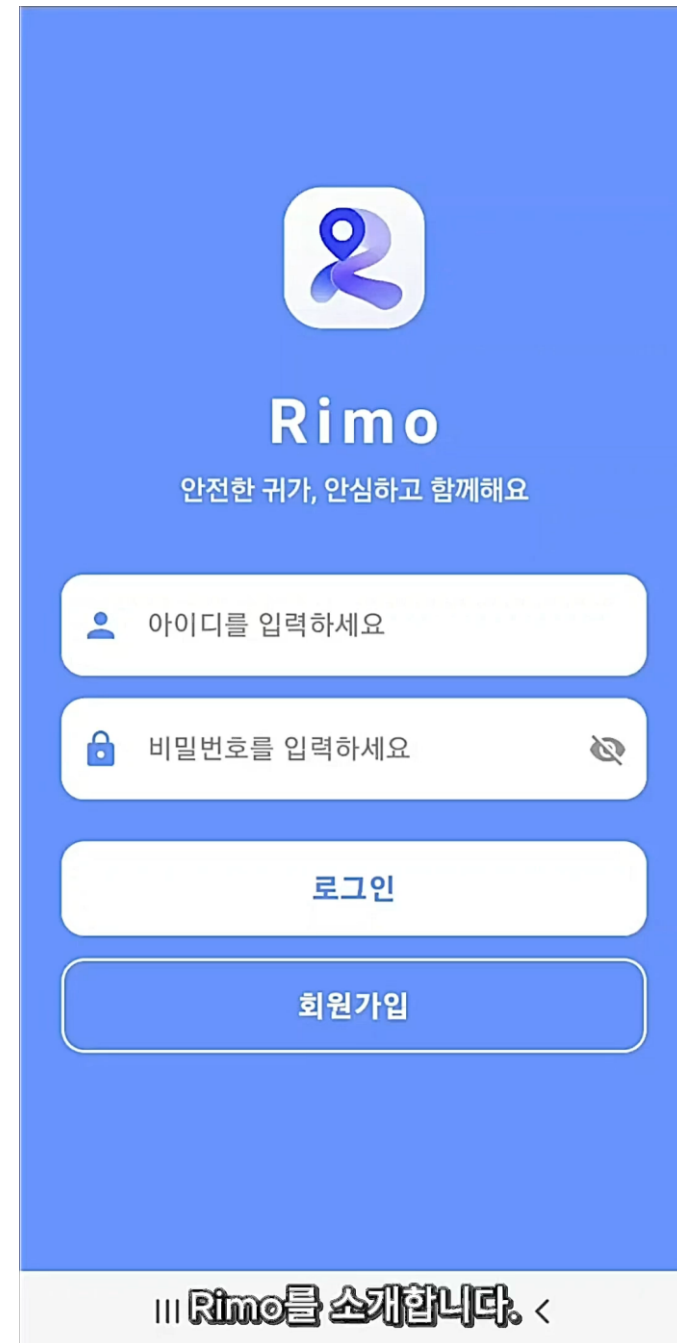
서비스 시연 영상

QR 코드를 스캔해 RIMO 앱을 다운로드하고 직접 사용해 보세요.

<https://www.rimo-app.com/>



다운로드 버튼과 QR 코드는 RIMO 공식 Android 설치 페이지로 연결됩니다.



SECTION 03

시스템 아키텍처

안정적인 서비스 구조

마이크로서비스와 AWS 인프라가 연결되는 전체 흐름을 설명합니다.



전체 기술 스택

RIMO 서비스에 적용한 기술과 외부 API

App



Android

Web Server · DB



Nginx



Spring Boot



MariaDB

AWS 네트워크 · 보안



Amazon VPC



Route 53



AWS WAF



ALB

컨테이너 · 컴퓨팅



Docker



Amazon ECR



Amazon EKS



Amazon EC2

스토리지 · 캐시



Amazon ElastiCache



Amazon S3

모니터링



Prometheus



Grafana



Amazon CloudWatch

CI/CD · IaC



GitHub



Jenkins



Terraform

외부 연동 API



Kakao Maps API



Kakao Mobility Directions API



SOLAPI



Gemini API

공공 데이터



공공데이터 API

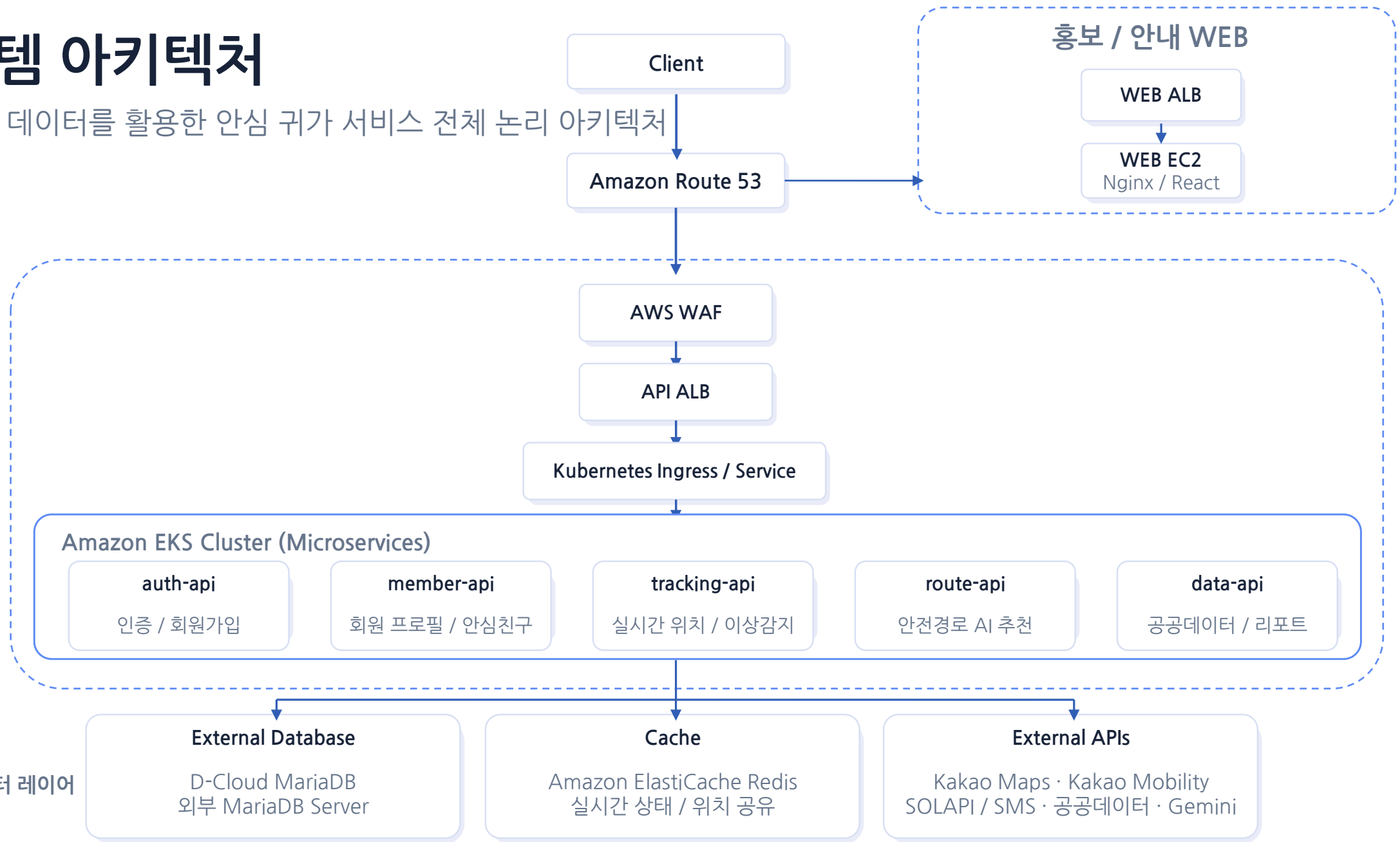
시스템 아키텍처

안전시설 데이터를 활용한 안심 귀가 서비스 전체 논리 아키텍처

API 서비스

마이크로서비스

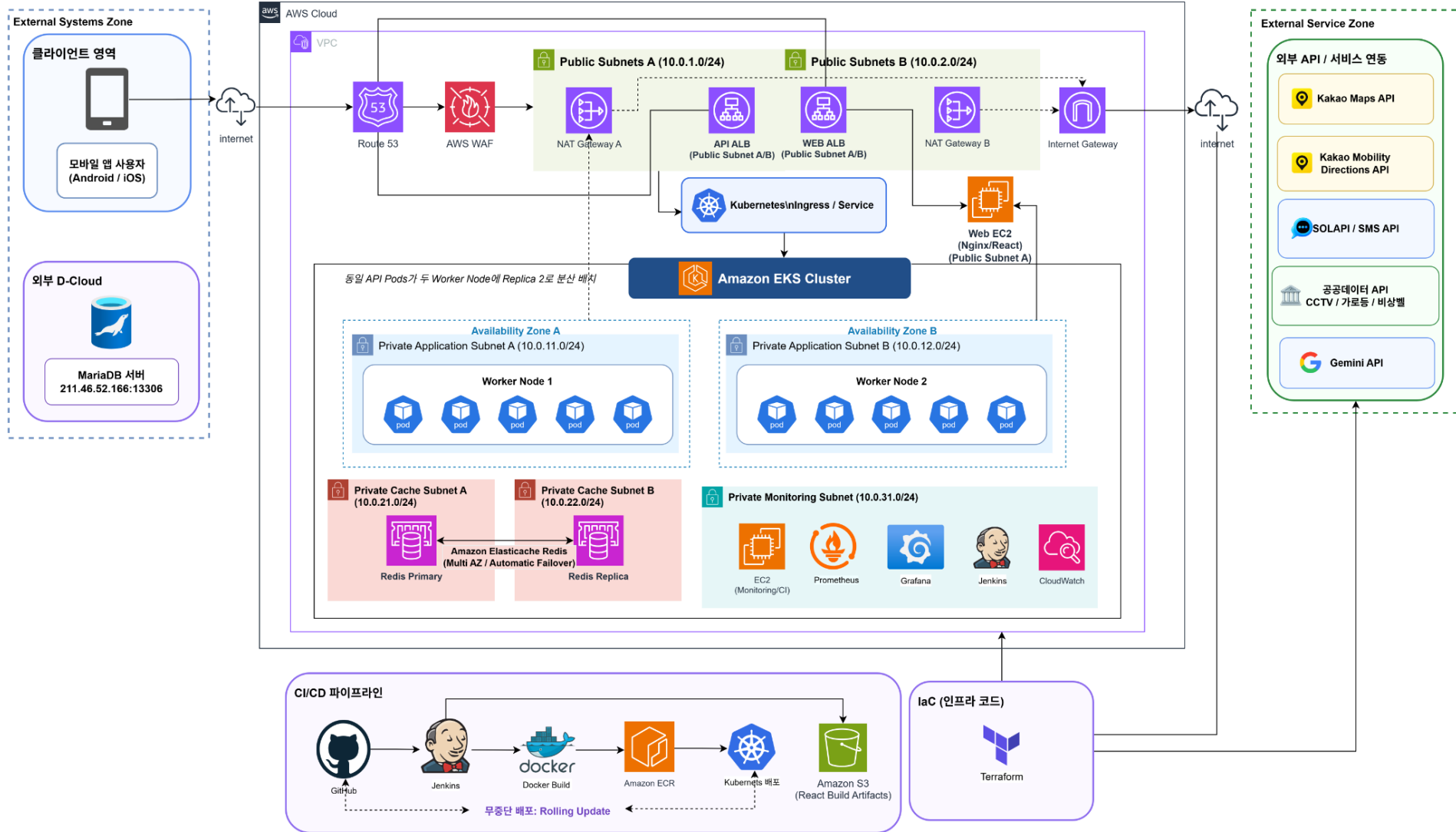
외부 연계 및 데이터 레이어



AWS 인프라 토폴로지

Multi-AZ(ap-northeast-2a/c) 기반 고가용성 네트워크 및 VPC 서브넷 구성

안심귀가 서비스 AWS 인프라 토폴로지 (EKS / 이중화 / 무중단 배포)



SECTION 04

시스템 구성 및 운영

운영 가능한 시스템

자동 배포, 실시간 관측, 부하 대응과 코드 기반 인프라를 검증합니다.



EKS 고가용성 및 Auto Scaling 구성

2개 가용영역에 API Pod를 분산하고 부하와 장애에 대응하는 서비스 운영 구조



구성 목적

- 긴급 기능은 짧은 중단도 사용자 신뢰에 영향을 줌
실시간 위치 공유와 SOS 요청이 특정 Node 장애로 멈추지 않아야 함
- 위치 갱신과 알림이 한 시점에 집중될 수 있음
고정된 Pod 수만으로 운영하면 응답 지연과 요청 실패 가능성이 커짐

운영 효과

- 2AZ와 Replica 분산으로 단일 장애 지점 완화
한 Worker Node에 문제가 생겨도 다른 Node의 Pod가 요청을 계속 처리
- Self-Healing과 HPA로 장애 복구와 부하 대응 자동화
실패한 Pod를 다시 생성하고, 부하가 증가하면 Pod 수를 확장

마이크로서비스(MSA) 분리 및 컨테이너 보안

5개 API 독립 구성과 ECR 자동 취약점 검사를 통한 서비스 안정성 강화

문제

단일 장애 전파

위치 추적 등 부하 발생 시 전체 서비스 중단 위험

해결

5개 API 독립 구성

역할별 독립 컨테이너 분리로 상호 영향 격리

결과

서비스 생존성 극대화

국지적 문제 발생 시에도 핵심 기능 정상 작동

문제

취약점 유입 위험

검증되지 않은 이미지가 운영 환경(EKS)에 배포될 위험

해결

배포 전 자동 보안 검사

ECR Push 시 자동 취약점 스캔 적용

결과

안전한 배포 환경 구축

사전 검증된 코드만 반영되어 전체 방어력 향상

EKS 모니터링 및 최소 권한 접근 제어

CloudWatch와 Access Entry를 활용한 운영 가시성 및 보안 확보



통합 모니터링 구축

한계 분산된 컨테이너 환경의 에러 추적 난해

해결 CloudWatch 기반 로그 중앙 집중화

결과 실시간 로그 분석 및 즉각적 장애 대응



최소 권한 제어 원칙

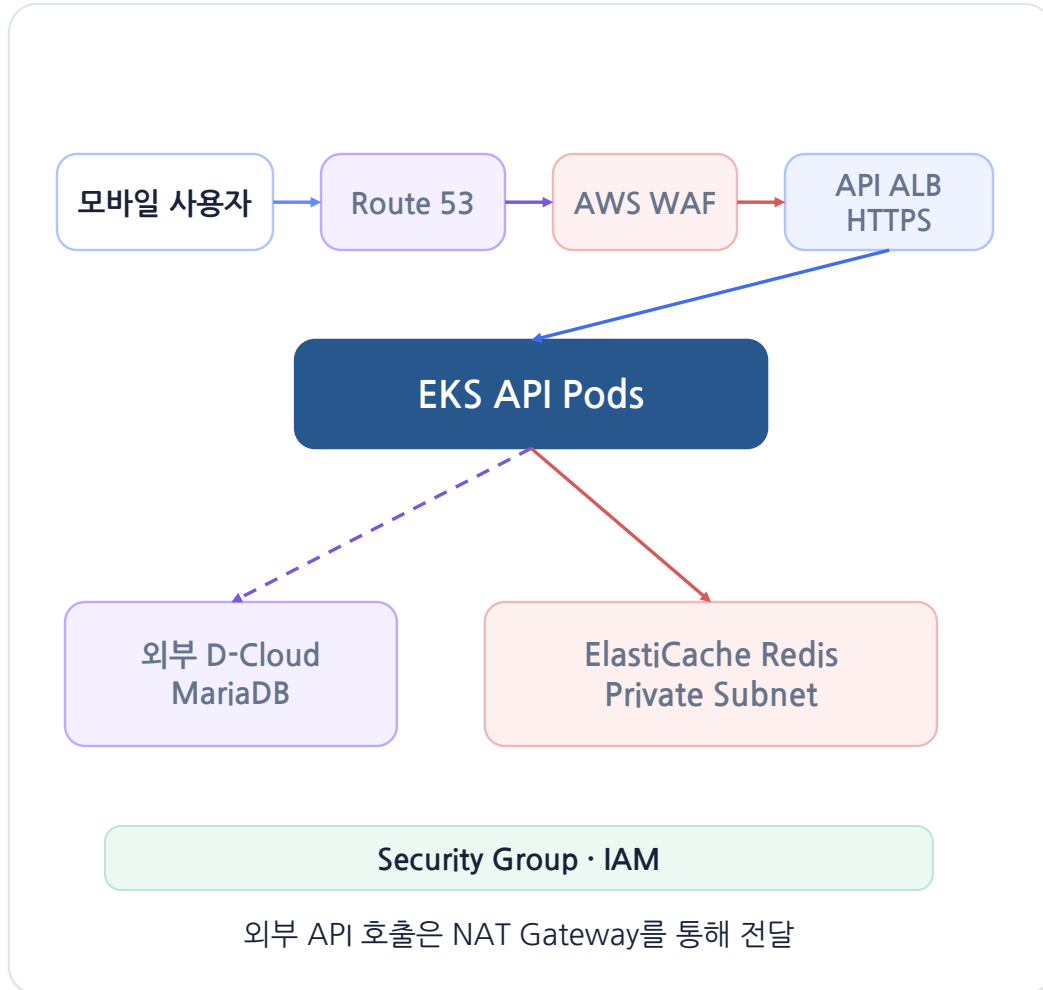
한계 모니터링 도구의 과도한 제어 권한 탈취 위협

해결 Access Entry를 통한 읽기 전용 권한 허용

결과 클러스터 보안 통제력과 관측성 동시 달성

Security / DNS&HTTPS

외부 D-Cloud 데이터와 AWS 진입 구간을 분리하고, Redis와 보안 계층으로 서비스 안정성 확보



구성 목적

- 외부 공격 표면 최소화 및 다계층 보안 구축

인터넷 진입 구간(Route 53, WAF, ALB)과 내부 서비스 영역(EKS, DB)을 Public/Private으로 엄격히 분리하여 인프라 직접 접근을 차단합니다.

- 통신 구간의 안전성 보장

클라이언트부터 진입점(ALB)까지 HTTPS 암호화 통신을 강제하여, 전송 중인 민감한 데이터의 스니핑 및 위변조를 방지합니다.

운영 효과

- 웹 취약점 및 비정상 트래픽 사전 차단

WAF를 통해 비정상 API 호출을 내부 애플리케이션 도달 전 최전방에서 필터링

- 내부 인프라 격리로 데이터 보호 강화

핵심 자원을 Private Subnet에 배치하고 외부 연동은 NAT Gateway로만 허용

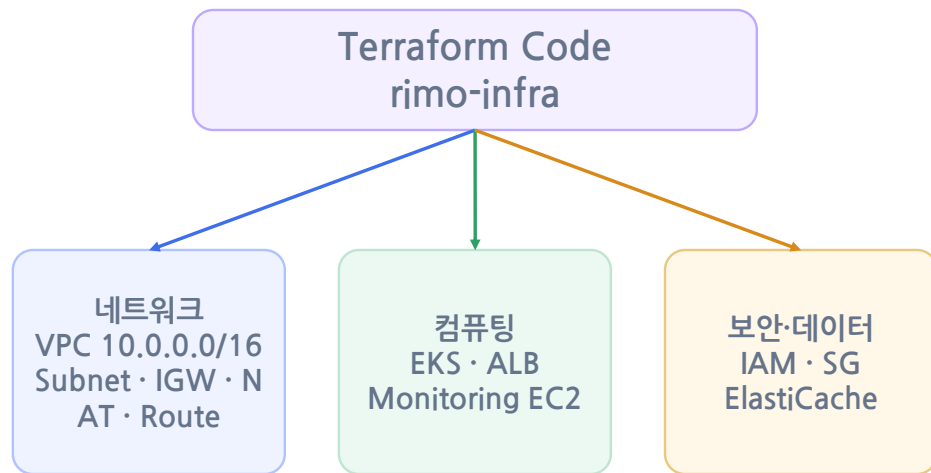
- 중앙 집중식 인증서 관리

ALB에서 도메인 및 HTTPS 인증서를 통합 처리하여 내부 서버의 암호화 연산 부하를 줄이고 보안 운영관리를 단순화

Terraform 기반 인프라 구성 관리

2AZ·EKS·네트워크 토폴로지를 코드로 관리하는 팀 인프라 운영 방식

관리 범위



실행 흐름



외부 D-Cloud DB와 Kakao·SOLAPI·Gemini API는 연동 대상이며 Terraform 관리 범위와 분리

구성 목적

- 2AZ 토폴로지는 연결 관계와 생성 순서가 복잡함
Subnet, Route Table, NAT, Security Group 설정이 하나만 빠져도 EKS와 외부 통신에 문제가 발생
- 콘솔 수동 작업은 팀원마다 설정이 달라질 수 있음
CIDR, 태그, 권한 설정을 코드에 고정해 동일한 기준으로 검토할 필요가 있음

운영 효과

- plan으로 변경 범위를 먼저 확인
실제 리소스를 만들기 전에 생성·수정·삭제 대상을 검토해 실수 가능성을 낮춤
- apply로 동일한 구성을 반복 적용
VPC와 EKS 관련 설정을 코드 기준으로 재현해 수동 누락과 환경 차이를 줄임
- Git 이력으로 변경 원인과 담당 범위를 추적
인프라 변경 내용을 코드 리뷰 대상으로 남겨 팀 운영의 투명성을 높임

Jenkins 기반 CI/CD 파이프라인 구축

main 브랜치 변경을 감지해 5개 백엔드 서비스를 Amazon EKS에 자동 배포

자동 배포 흐름



직접 구현한 내용

- Jenkinsfile에 ECR Push와 EKS 배포 단계 추가
GitHub main 체크아웃 이후 빌드부터 배포까지 한 파일에서 실행
- 백엔드 5개 서비스를 각각 이미지로 빌드
auth, data-api, member-api, route-api, tracking-api
- kubectl로 이미지 교체 후 rollout 상태 확인
실행 순서: aws eks update-kubeconfig, kubectl set image, rollout status

문제 해결과 결과

- 권한 오류 EKS Deployment patch 권한 부족
EditPolicy 추가 후 배포 성공
- 최종 결과 5개 백엔드 서비스의 Build·ECR
Push·EKS Rolling Update를
하나의 Jenkins Pipeline으로 통합

Monitoring - 실제 Grafana Dashboard

Prometheus와 Grafana를 활용한 EKS/Pod 메트릭 실시간 수집 및 시각화



메트릭 데이터 수집

EKS·Node·Pod 메트릭 수집
CPU·Memory·Pod 수 추적

실시간 시각화 대시보드

Grafana로 데이터 시각화
운영 상태를 한눈에 확인

핵심 지표 집중 모니터링

CPU·Memory·HPA 추이 확인
서비스 이상 징후 탐지

Prometheus와 Grafana로 EKS/Pod 운영 상태를 실시간 확인합니다.

RIMO Observability 모니터링 체계

서비스·캐시·AWS 인프라·로그를 통합 관측하는 운영 환경

4개의 목적별 대시보드

Backend Monitoring

5개 API CPU·Memory·Pod 수
응답시간과 외부 API 호출

AWS Infra Monitoring

CloudWatch 연동
ALB·NAT·WAF·EKS Node 상태

Redis Monitoring

Cache Hit Rate·메모리
Command Throughput

Developer Monitoring

Loki·Promtail 로그 수집
Error Log·Jenkins 빌드 상태



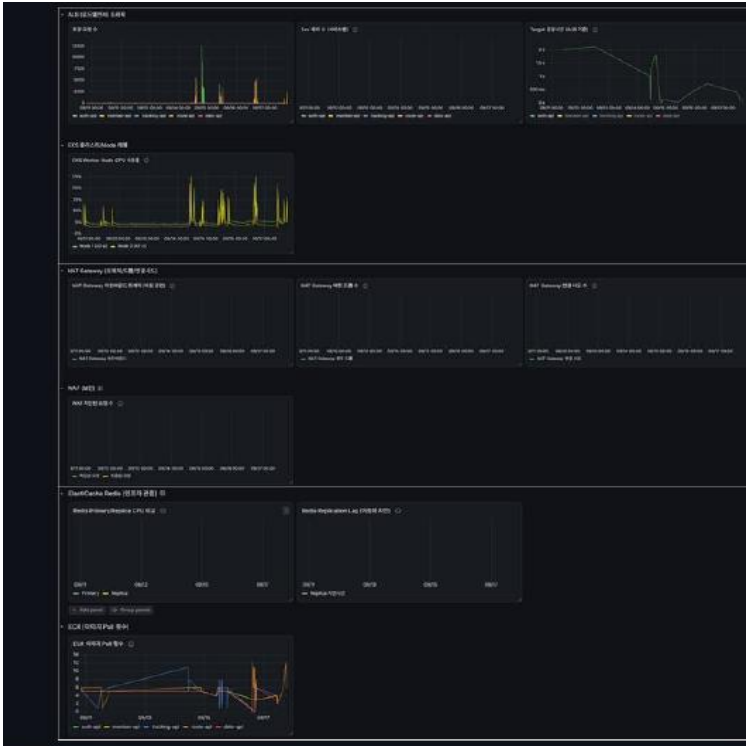
서비스·캐시·AWS 인프라·로그를 함께 관측합니다.

RIMO Observability 모니터링 체계

Redis · AWS 인프라 · Developer Monitoring 화면

AWS Infra Monitoring

ALB · NAT Gateway · WAF · EKS Node 지표를
통합 모니터링



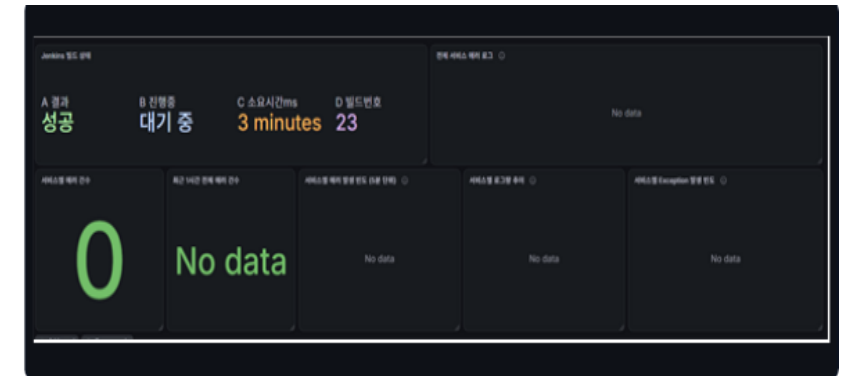
Redis Monitoring

Cache Hit Rate, Memory,
Throughput 등 Redis 상태 확인



Developer Monitoring

Jenkins 빌드 상태와 서비스
로그·에러 현황 확인



세 대시보드로 Redis, AWS, 개발 상태를 함께 점검합니다.

이상 징후, 바로 알아챌 수 있을까?

Grafana 임계값 알림과 Slack 연동으로 장애 상황을 실시간으로 인지합니다

1 대시보드가 빨간색으로 바뀝니다

route-api CPU 사용률이 임계값(70%)을 넘으면
Grafana 패널 색상이 자동으로 붉게 표시됩니다.



2 Slack으로 알림이 도착합니다

동시에 Grafana Alerting이 Slack #rimo-alerts 채널로
알림을 실시간 전송합니다.



RIMO Monitoring 오전 11:52

[FIRING:1] route-api CPU 급상승 알림 RIMO 알림 규칙

Firing

Value: A=0.8324692550206637, C=1

Labels:

- alertname = route-api CPU 급상승 알림

- grafana_folder = RIMO 알림 규칙

[자세히 표시](#)

Grafana v13.2.1 | 오늘 오전 11:52

[RESOLVED] route-api CPU 급상승 알림 RIMO 알림 규칙

Resolved

Value: A=0.027014116175758315, C=0

Labels:

- alertname = route-api CPU 급상승 알림

- grafana_folder = RIMO 알림 규칙

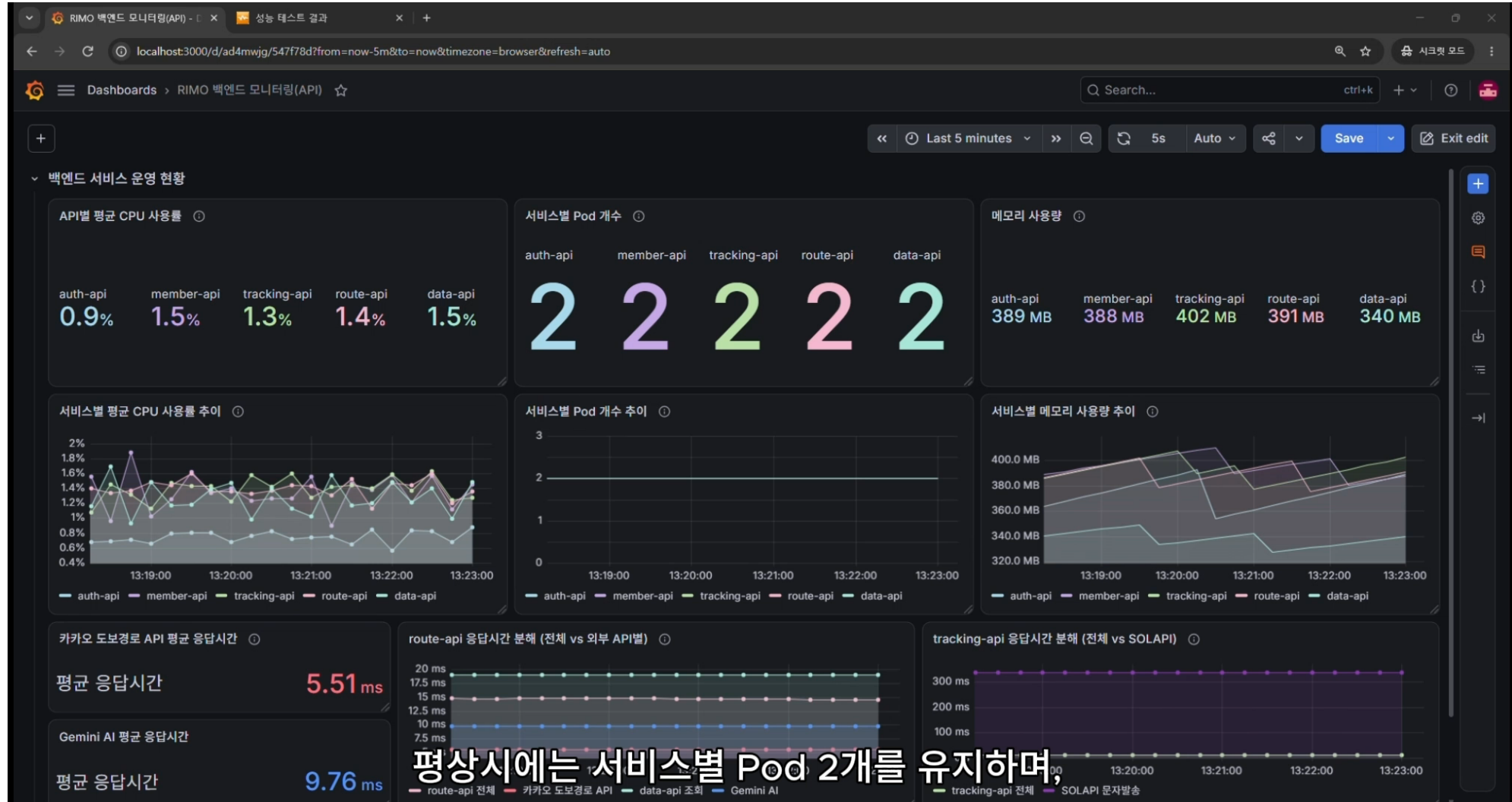
[자세히 표시](#)

Grafana v13.2.1 | 오늘 오전 11:57

서비스·캐시·AWS 인프라·로그를 함께 관측합니다.

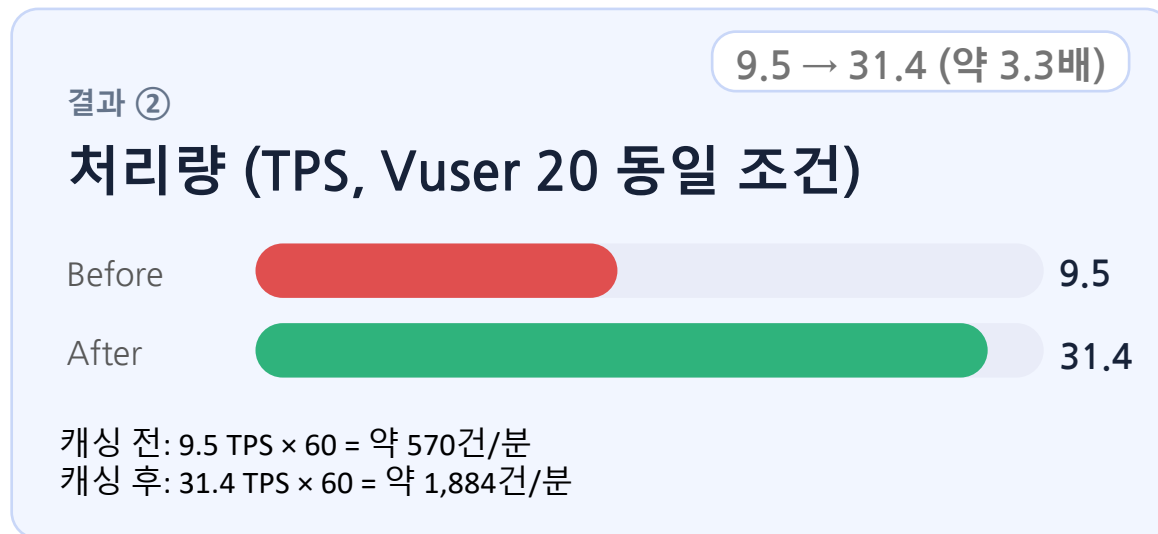
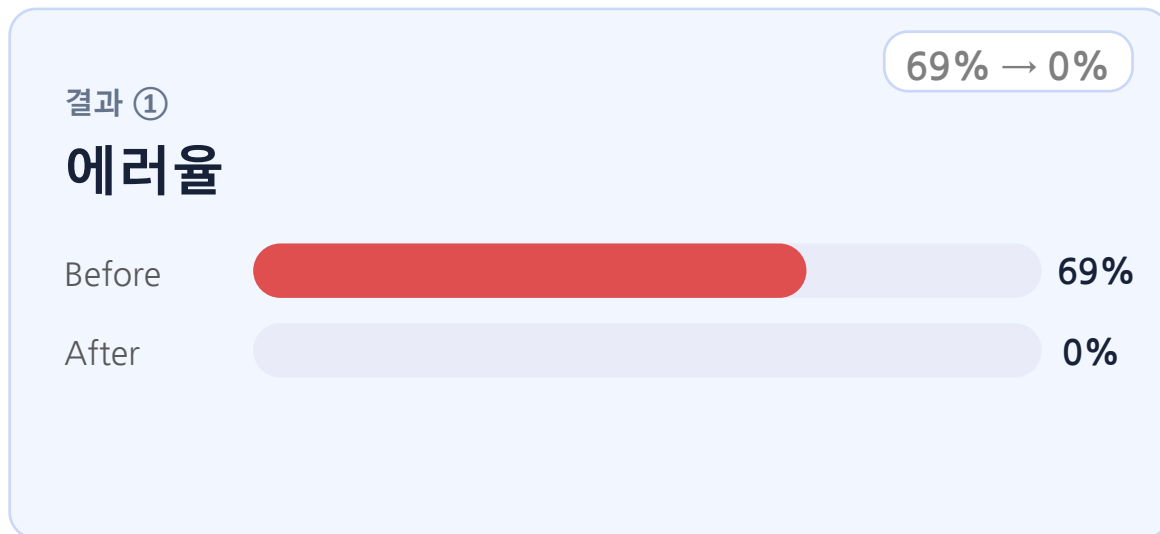
부하테스트 영상

부하 증가 시 Pod 자동 확장(2 → 8) 실시간 검증



카카오 API 병목 해결

에러율 69% → 0%, 캐싱으로 외부 API 호출 자체를 줄인 결과



카카오 API 호출을 Redis로 캐싱해, 에러를 없애고 처리량을 약 3.3배 끌어올렸습니다.

Gemini 응답 지연 해결

Pod 8개로 확장해도 남아있던 지연, 캐싱으로 근본 해결

Pod를 2개 → 8개까지 늘렸지만, CPU 부하는 300~500%대로 그대로 유지

원인: Pod 개수가 아니라, 외부 API(Gemini) 응답을 기다리는 시간

문제

처리 속도

Pod 8개에도 부하 유지

원인

Gemini AI 응답 대기

처리 시간의 약 58%

해결

Gemini 캐싱

같은 조건의 추천 문구 재사용

결과

처리량 (TPS, Vuser 100 동일 조건)

30.0 → 34.0 (+13.3%)

Before



30.0

After



34.0

캐싱 전: $30.0 \text{ TPS} \times 60 = \text{약 } 1,800\text{건/분}$

캐싱 후: $34.0 \text{ TPS} \times 60 = \text{약 } 2,040\text{건/분}$

확장 방식 보완

1. Pod 확장은 기본, 외부 API 병목은 캐싱으로

- CPU + 요청량 기준 Pod 확장
- 적은 요청에도 부하가 빠르게 오르는 특성 반영

2. Karpenter로 Node 자동 추가

- Pod 배치 공간 부족 시 Worker Node 자동 증설

Pod 확장은 기본, 외부 API 병목은 호출 자체를 줄여야 늘어난 Pod가 의미 있습니다.

SECTION 05

결론 및 팀 소개

프로젝트를 완성한 사람들

기술 구성과 기대효과를 정리하고 팀의 역할과 기여를 소개합니다.



프로젝트 요약 및 기대효과

결론



안심 귀가 경험

안전시설 기반 경로와
실시간 보호 기능을 통해

이동 과정 전반의 안전 지원

“

RIMO는 안전시설 데이터,
실시간 위치 공유 및 이상 상황 대응,
Cloud Native 인프라를
하나의 귀가 과정으로 연결한
안전 귀가 지원 서비스를 구현했습니다.



안정적인 서비스 기반

AWS/EKS 기반 이중화와
자동화 환경을 통해

서비스 안정성 및 확장성 확보

팀원 소개

역할과 기여 내용

맹시윤 (팀장)



- 모바일 UI/UX 개발
- 안심경로/대시보드 개발
- Terraform·EKS 인프라 구축
- HPA 트래픽 대응 구성
- RIMO 웹페이지 개발·배포

이동기



- 테스트 환경·네트워크 구축
- API 통합 및 보안 적용
- 실시간 위치공유 개발
- 회원가입/로그인 개발

이수경



- 모니터링 구축
- Prometheus·Grafana 구축
- 부하 테스트
- 긴급신고·안전 알림 개발

임수현



- Jenkins CI/CD 구축
- 공공데이터 수집
- 친구 관리 기능 개발
- 지도·알림 기능 개발

Q&A

경청해 주셔서 감사합니다.

궁금하신 점은 편하게 질문해 주세요.

참고자료

프로젝트 기획 및 시스템 구축 주요 참고 문헌

- 01 통계청, 「2024년 사회조사」
- 02 서울특별시, 서울 안심이 및 안심헬프미 공식자료
- 03 AWS, EKS, Kubernetes, Terraform Official Documentation
- 04 Prometheus, Grafana Official Documentation